

```
# =====
# BLOB Principe - SAUT 3.5 v3 : EXTENSION GRANDE ECHELLE N=5000
# =====
#
# Objectif : completer Saut 3.5 v2 en atteignant les paliers N=1000, 2000, 5000
# pour confirmer asymptotiquement les lois d echelle :
#   PHI ~ N^1.35  (mesure v2 sur N=50-500, R2=0.98)
#   n_tetra ~ N^1.78  (mesure v2 sur N=50-500, R2=0.99)
#
# Architecture optimisee :
#   - Demarrage N=1, arret N=5000
#   - Mesures dorenavant aux paliers : 100, 200, 500, 1000, 2000, 5000
#     (skip N=50 qui est tres bruite)
#   - 3 seeds par mode (BLOB / NULL)
#   - PHI calcul lourd seulement aux paliers (pas continuellement)
#   - Sauvegarde intermediaire apres chaque seed (pour ne rien perdre)
#
# Tests automatiques :
#   T1 - PHI ~ N^1.35 sur 6 paliers (R2 > 0.98)
#   T2 - n_tetra ~ N^1.78 sur 6 paliers (R2 > 0.98)
#   T3 - r3 -> 4.65 a N=5000 (CV < 5% sur 3 seeds)
#   T4 - Q3 Blob / Q3 Null tend vers une asymptote stable
#   T5 - PHI Null reste a 0 (confirmation Blob-specifique)
# =====
```

```
import sys
```

```
import os
```

```
def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous = {chr(8216): "U+2018", chr(8217): "U+2019",
                chr(8220): "U+201C", chr(8221): "U+201D",
                chr(8211): "U+2013", chr(8212): "U+2014"}
            found = [(name, content.count(c)) for c, name in dangerous.items() if
```

```

        if found:
            print("ERREUR caracteres corrompus :", found)
            return False
        print("OK : pas de corruption")
    else:
        print("Note : auto-test partiel")
except Exception as e:
    print("Note : " + str(e))
print("=" * 70)
return True

if not auto_check():
    sys.exit(1)

import numpy as np
import time
import json
import gc
from collections import Counter

KAPPA_0 = 8.0
BETA_VIAB = 0.30
TAU = 0.50
P_MITOSE = 0.001
EPS_DECAL = 0.05

OUTPUT_DIR = '/content/blob_saut35v3_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

class BlobUniverseGrowing:
    """Demarre N=1, croit par mitose, mode 'blob' ou 'null'."""

    def __init__(self, seed=0, mode='blob'):
        self.rng = np.random.default_rng(seed)
        self.seed = seed
        self.mode = mode
        self.positions = [np.array([0.5, 0.5, 0.5])]
        self.adj = [set()]
        self.deg = np.array([0], dtype=np.int32)
        self.edge_list = []
        self.n_tri = 0
        self.n_tetra = 0
        self.S_curr = 0.0
        self.Q3 = 0.0
        self.n_mitoses = 0

```

```

        self.n_swaps_accepted = 0
        self.first_edge_step = None
        self.first_triangle_step = None
        self.first_tetrahedron_step = None

    @property
    def N(self):
        return len(self.positions)

    @property
    def E(self):
        return len(self.edge_list)

    def _compute_D(self):
        if self.N == 0:
            return 0.0
        mu = float(self.deg.mean())
        if mu == 0:
            return 0.0
        return float(self.deg.std()) / mu

    def _compute_Q1(self):
        if self.N == 0:
            return 0.0
        c = Counter(self.deg.tolist())
        ent = 0.0
        for cnt in c.values():
            p = cnt / self.N
            if p > 0:
                ent -= p * np.log(p)
        return float(ent)

    def _rips_radius(self):
        if self.N <= 1:
            return 0.5
        c_3 = 4 * np.pi / 3
        return 1.5 * (KAPPA_0 / (c_3 * self.N)) ** (1.0/3)

    def _count_tetra_through_edge(self, u, v):
        common = self.adj[u] & self.adj[v]
        if len(common) < 2:
            return 0
        cl = list(common)
        count = 0
        for i, w in enumerate(cl):
            adj_w = self.adj[w]
            for x in cl[i+1:]:

```

```

        if x in adj_w:
            count += 1
    return count

def _list_tetrahedra(self):
    result = []
    seen = set()
    for u, v in self.edge_list:
        common = self.adj[u] & self.adj[v]
        if len(common) < 2:
            continue
        cl = sorted(common)
        for i, w in enumerate(cl):
            for x in cl[i+1:]:
                if x in self.adj[w]:
                    key = tuple(sorted([u, v, w, x]))
                    if key not in seen:
                        seen.add(key)
                        result.append(key)
    return result

def compute_PHI(self):
    if self.n_tetra == 0:
        return 0.0
    tetras = self._list_tetrahedra()
    if not tetras:
        return 0.0
    edge_to_tetras = {}
    for tet in tetras:
        for i in range(4):
            for j in range(i+1, 4):
                e = (tet[i], tet[j])
                edge_to_tetras.setdefault(e, []).append(tet)
    neighbors_count = {tet: set() for tet in tetras}
    for e, tlist in edge_to_tetras.items():
        if len(tlist) > 1:
            for i, t1 in enumerate(tlist):
                for t2 in tlist[i+1:]:
                    neighbors_count[t1].add(t2)
                    neighbors_count[t2].add(t1)
    total = sum(len(v) for v in neighbors_count.values())
    return float(total) / len(tetras)

def compute_PSI(self):
    if self.N == 0:
        return 0.0
    total_ent = 0.0

```

```

n_active = 0
for v in range(self.N):
    neighbors = self.adj[v]
    if not neighbors:
        continue
    n_active += 1
    local_degs = [self.deg[u] for u in neighbors]
    counts = Counter(local_degs)
    total = len(local_degs)
    ent = 0.0
    for c in counts.values():
        p = c / total
        if p > 0:
            ent -= p * np.log(p)
    total_ent += ent
return float(total_ent / n_active) if n_active > 0 else 0.0

def mitose(self):
    parent = int(self.rng.integers(self.N))
    parent_pos = self.positions[parent]
    r_p = self._rips_radius()
    decal = self.rng.normal(0, EPS_DECAL * r_p, size=3)
    new_pos = parent_pos + decal
    self.positions.append(new_pos)
    self.adj.append(set())
    self.deg = np.append(self.deg, 0)
    self.n_mitoses += 1

def step(self, current_step):
    if self.rng.random() < P_MITOSE:
        self.mitose()
        return False
    if self.N < 2:
        return False
    if self.E == 0:
        return self._step_add_only(current_step)
    E_target = int(KAPPA_0 * self.N / 2)
    if self.E < E_target:
        deficit = (E_target - self.E) / max(E_target, 1)
        if self.rng.random() < deficit:
            return self._step_add_only(current_step)
    return self._step_swap(current_step)

def _step_add_only(self, current_step):
    u = int(self.rng.integers(self.N))
    v = int(self.rng.integers(self.N))
    if u == v or v in self.adj[u]:

```

```

        return False
    u, v = min(u, v), max(u, v)
    dt_add = len(self.adj[u] & self.adj[v])
    self.adj[u].add(v)
    self.adj[v].add(u)
    self.deg[u] += 1
    self.deg[v] += 1
    new_n_tri = self.n_tri + dt_add
    dtetra_add = self._count_tetra_through_edge(u, v)
    new_S = np.log(1 + new_n_tri) - BETA_VIAB * self._compute_D()
    delta_S = new_S - self.S_curr

    if self.mode == 'null':
        accepted = self.rng.random() < 0.5
    else:
        log_alpha = self.N * delta_S / TAU
        accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_al

if accepted:
    self.edge_list.append((u, v))
    self.n_tri = new_n_tri
    self.n_tetra += dtetra_add
    self.S_curr = new_S
    self.n_swaps_accepted += 1
    self.Q3 += abs(delta_S)
    if self.first_edge_step is None and self.E > 0:
        self.first_edge_step = current_step
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step
    return True
else:
    self.adj[u].discard(v)
    self.adj[v].discard(u)
    self.deg[u] -= 1
    self.deg[v] -= 1
    return False

def _step_swap(self, current_step):
    idx = int(self.rng.integers(len(self.edge_list)))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)

```

```

if (u_add, v_add) == (u_rm, v_rm):
    return False
dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._count_tetra_through_edge(u_add, v_add)
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
new_S = np.log(1 + new_n_tri) - BETA_VIAB * self._compute_D()
delta_S = new_S - self.S_curr

if self.mode == 'null':
    accepted = self.rng.random() < 0.5
else:
    log_alpha = self.N * delta_S / TAU
    accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_al

if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S
    self.n_swaps_accepted += 1
    self.Q3 += abs(delta_S)
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step
    return True
else:

```

```

        self.adj[u_add].discard(v_add)
        self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False

```

```

def measure_full_state(sim, step, t_start):
    """Mesure complete (PHI cher : ne le faire qu'aux paliers)."""
    return {
        'step': step,
        'N': sim.N, 'E': sim.E,
        'n_tri': sim.n_tri, 'n_tetra': sim.n_tetra,
        'D': sim._compute_D(),
        'Q1': sim._compute_Q1(),
        'PHI': sim.compute_PHI(),
        'PSI': sim.compute_PSI(),
        'Q3': sim.Q3,
        'r3': sim.n_tetra / sim.n_tri if sim.n_tri > 0 else 0.0,
        'log_ntetra_over_log_N': np.log(sim.n_tetra) / np.log(sim.N) if sim.N > 1
        'PHI_over_ntetra': sim.compute_PHI() / sim.n_tetra if sim.n_tetra > 0 else
        'time_total_s': time.time() - t_start,
    }

```

```

def run_one(seed, mode, N_target, paliers_N, max_steps=20_000_000):
    """Demarre N=1, mesure aux paliers, arret a N_target."""
    t_start = time.time()
    sim = BlobUniverseGrowing(seed=seed, mode=mode)
    snapshots = []
    next_palier_idx = 0

    print(f'    [{mode}-s{seed}] start...', flush=True)

    for step in range(max_steps):
        sim.step(step + 1)

        if next_palier_idx < len(paliers_N) and sim.N >= paliers_N[next_palier_idx]:
            palier_N = paliers_N[next_palier_idx]
            state = measure_full_state(sim, step + 1, t_start)
            state['palier_N'] = palier_N
            snapshots.append(state)
            print(f'    [{mode}-s{seed}] N={palier_N}: step={step+1} n_tetra={sim

```

```

        next_palier_idx += 1

        # Sauvegarde intermediaire systematique (securite)
        interim_path = OUTPUT_DIR + f'/interim_{mode}_seed{seed}.json'
        with open(interim_path, 'w') as f:
            json.dump({'mode': mode, 'seed': seed, 'snapshots': snapshots}, f)

    if sim.N >= N_target:
        break

    tri_recount = sum(len(sim.adj[u] & sim.adj[v]) for u, v in sim.edge_list) //
    coherent = (sim.n_tri == tri_recount)

    return {
        'seed': seed, 'mode': mode,
        'final_step': step + 1,
        'final_state': {
            'N': sim.N, 'E': sim.E,
            'n_tri': sim.n_tri, 'n_tetra': sim.n_tetra,
            'D': sim._compute_D(),
            'PHI': sim.compute_PHI(),
            'Q1': sim._compute_Q1(),
            'PSI': sim.compute_PSI(),
            'Q3': sim.Q3,
        },
        'first_edge_step': sim.first_edge_step,
        'first_triangle_step': sim.first_triangle_step,
        'first_tetrahedron_step': sim.first_tetrahedron_step,
        'tri_recount': tri_recount,
        'coherent': coherent,
        'snapshots': snapshots,
        'total_time_s': time.time() - t_start,
    }

# =====
# MAIN
# =====
print('')
print('=' * 75)
print(' BLOB - SAUT 3.5 v3 - EXTENSION GRANDE ECHELLE N=5000')
print('=' * 75)

N_TARGET = int(os.environ.get('BLOB_N_TARGET', 5000))
N_SEEDS = int(os.environ.get('N_SEEDS', 3))
RUN_NULL = os.environ.get('RUN_NULL', '1') == '1'
MAX_STEPS = int(os.environ.get('MAX_STEPS', 20_000_000))

```

```

# Paliers : log-spaced, plus de points a grand N
PALIERS_N = [100, 200, 500, 1000, 2000, 5000]
PALIERS_N = [p for p in PALIERS_N if p <= N_TARGET]

print(f' N_target : {N_TARGET}')
print(f' Paliers N : {PALIERS_N}')
print(f' Seeds : {N_SEEDS}')
print(f' Run NULL : {RUN_NULL}')
print(f' Max steps : {MAX_STEPS}')
n_modes = 2 if RUN_NULL else 1
estim_min = N_SEEDS * n_modes * (N_TARGET / 500) * 1.5
print(f' Estimation duree : ~{estim_min:.0f} min')
print('=' * 75)

t_global = time.time()
results_blob = []
results_null = []

print('')
print(' === MODE BLOB ===')
for seed in range(N_SEEDS):
    print(f'')
    print(f' --- Seed {seed} BLOB ---')
    r = run_one(seed, 'blob', N_TARGET, PALIERS_N, MAX_STEPS)
    results_blob.append(r)

    # Sauvegarde apres chaque seed
    inter_path = OUTPUT_DIR + '/saut35v3_RAPPORT_partiel.json'
    with open(inter_path, 'w') as f:
        json.dump({'blob': results_blob, 'null': results_null}, f, default=str)

gc.collect()

if RUN_NULL:
    print('')
    print(' === MODE NULL ===')
    for seed in range(N_SEEDS):
        print(f'')
        print(f' --- Seed {seed} NULL ---')
        r = run_one(seed, 'null', N_TARGET, PALIERS_N, MAX_STEPS)
        results_null.append(r)

    inter_path = OUTPUT_DIR + '/saut35v3_RAPPORT_partiel.json'
    with open(inter_path, 'w') as f:
        json.dump({'blob': results_blob, 'null': results_null}, f, default=st

```

```

gc.collect()

total_time = time.time() - t_global
print('')
print('=' * 75)
print(f' Toutes simulations terminees en {total_time/60:.1f} min')
print('=' * 75)

# =====
# AGREGATION ET ANALYSE
# =====
def aggregate(results):
    agg = {}
    for r in results:
        for snap in r['snapshots']:
            p = snap['palier_N']
            if p not in agg:
                agg[p] = {k: [] for k in ['PHI', 'PSI', 'Q1', 'D', 'r3',
                                           'log_ntetra_over_log_N', 'PHI_over_nte',
                                           'n_tri', 'n_tetra', 'Q3']}
            for k in agg[p]:
                if k in snap:
                    agg[p][k].append(snap[k])
    return agg

blob_agg = aggregate(results_blob)
null_agg = aggregate(results_null) if RUN_NULL else {}

paliers_sorted = sorted(blob_agg.keys())

# =====
# AFFICHAGE PROFIL BLOB
# =====
print('')
print(' --- PROFIL BLOB par palier N ---')
print(f' {"N":>5s} {"PHI":>10s} {"PSI":>8s} {"Q1":>8s} {"D":>8s} {"r3":>8s}')
print(' ' + '-' * 100)
for p in paliers_sorted:
    a = blob_agg[p]
    means = {k: np.mean(v) if v else 0 for k, v in a.items()}
    print(f' {p:>5d} {means["PHI"]:>10.3f} {means["PSI"]:>8.4f} {means["Q1"]:'

if RUN_NULL:
    print('')
    print(' --- PROFIL NULL par palier N ---')
    print(f' {"N":>5s} {"PHI":>10s} {"r3":>8s} {"n_tetra":>9s} {"D":>8s} {"')
    print(' ' + '-' * 60)

```

```

    for p in sorted(null_agg.keys()):
        a = null_agg[p]
        means = {k: np.mean(v) if v else 0 for k, v in a.items()}
        print(f'    {p:>5d}    {means["PHI"]:>10.3f}    {means["r3"]:>8.4f}    {means["n_

# =====
# TESTS T1-T5
# =====

print('')
print('=' * 75)
print('  TESTS DE VALIDATION')
print('=' * 75)

# T1 : PHI ~ N^1.35 sur 6 paliers
print('')
print('  T1 - PHI ~ N^alpha (par seed)')
all_alpha_phi = []
for r in results_blob:
    Ns = [s['N'] for s in r['snapshots']]
    PHIs = [s['PHI'] for s in r['snapshots']]
    if all(p > 0 for p in PHIs) and len(Ns) >= 4:
        log_N = np.log(Ns)
        log_phi = np.log(PHIs)
        slope, intercept = np.polyfit(log_N, log_phi, 1)
        pred = slope * log_N + intercept
        ss_res = np.sum((log_phi - pred)**2)
        ss_tot = np.sum((log_phi - log_phi.mean())**2)
        r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0
        print(f'      seed {r["seed"]}: alpha = {slope:.4f}, R2 = {r2:.4f}')
        all_alpha_phi.append(slope)

if all_alpha_phi:
    mean_alpha_phi = np.mean(all_alpha_phi)
    cv_alpha_phi = np.std(all_alpha_phi) / abs(mean_alpha_phi) * 100
    T1_pass = (cv_alpha_phi < 5.0)
    print(f'      >> alpha moyen = {mean_alpha_phi:.4f}, CV = {cv_alpha_phi:.1f}%,

# T2 : n_tetra ~ N^1.78
print('')
print('  T2 - n_tetra ~ N^beta (par seed)')
all_beta_nt = []
for r in results_blob:
    Ns = [s['N'] for s in r['snapshots']]
    nts = [s['n_tetra'] for s in r['snapshots']]
    if all(nt > 0 for nt in nts) and len(Ns) >= 4:
        log_N = np.log(Ns)
        log_nt = np.log(nts)

```

```

        slope, intercept = np.polyfit(log_N, log_nt, 1)
        pred = slope * log_N + intercept
        ss_res = np.sum((log_nt - pred)**2)
        ss_tot = np.sum((log_nt - log_nt.mean())**2)
        r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0
        print(f'        seed {r["seed"]}: beta = {slope:.4f}, R2 = {r2:.4f}')
        all_beta_nt.append(slope)

if all_beta_nt:
    mean_beta_nt = np.mean(all_beta_nt)
    cv_beta_nt = np.std(all_beta_nt) / abs(mean_beta_nt) * 100
    T2_pass = (cv_beta_nt < 5.0)
    print(f'        >> beta moyen = {mean_beta_nt:.4f}, CV = {cv_beta_nt:.1f}%, T2 =

# T3 : r3 -> 4.65 a N=5000
print('')
print(' T3 - r3 a N=5000 doit converger vers 4.65')
if 5000 in blob_agg:
    r3_finals = blob_agg[5000]['r3']
    r3_mean = np.mean(r3_finals)
    r3_cv = np.std(r3_finals) / abs(r3_mean) * 100
    T3_pass = (4.0 <= r3_mean <= 5.5) and (r3_cv < 5.0)
    print(f'        r3 a N=5000 : mean = {r3_mean:.4f}, CV = {r3_cv:.1f}%, T3 = {"PAS
else:
    print(f'        Pas de palier N=5000 atteint')
    T3_pass = False

# T4 : Q3 Blob/Q3 Null asymptote
print('')
print(' T4 - Q3 Blob/Null par palier (asymptote attendue)')
if RUN_NULL:
    for p in paliers_sorted:
        if p in null_agg:
            q3_b = np.mean(blob_agg[p]['Q3'])
            q3_n = np.mean(null_agg[p]['Q3'])
            ratio = q3_b / q3_n if q3_n > 0 else float('inf')
            print(f'        N={p}: Q3 Blob = {q3_b:.1f}, Q3 Null = {q3_n:.1f}, ratio

# T5 : PHI Null reste a 0
print('')
print(' T5 - PHI Null doit rester nul a grand N')
if RUN_NULL:
    for p in sorted(null_agg.keys()):
        phi_n = np.mean(null_agg[p]['PHI'])
        nt_n = np.mean(null_agg[p]['n_tetra'])
        flag = ''
        if p >= 200 and (phi_n > 0.1 or nt_n > 1):

```

```

        flag = '  <- inattendu'
    print(f'      N={p}: PHI Null = {phi_n:.4f}, n_tetra Null = {nt_n:.2f}{flag}

# =====
# SAUVEGARDE FINALE
# =====

report = {
    'config': {
        'N_target': N_TARGET, 'paliers_N': PALIERS_N,
        'n_seeds': N_SEEDS, 'run_null': RUN_NULL,
        'total_time_s': total_time,
    },
    'blob_aggregated': {p: {k: list(v) for k, v in d.items()} for p, d in blob_ag
    'null_aggregated': {p: {k: list(v) for k, v in d.items()} for p, d in null_ag
    'all_results_blob': results_blob,
    'all_results_null': results_null if RUN_NULL else [],
    'fits': {
        'PHI_alpha': all_alpha_phi if all_alpha_phi else [],
        'n_tetra_beta': all_beta_nt if all_beta_nt else [],
    },
}

report_path = OUTPUT_DIR + '/saut35v3_RAPPORT.json'
with open(report_path, 'w') as f:
    json.dump(report, f, indent=2, default=str)
print('')
print(f' Rapport final : {report_path}')
print(f' Taille : {os.path.getsize(report_path)} octets')

```